

1.a)

Priors:

$$P(C=A) = \frac{3}{7}$$

$$P(C=B) = \frac{4}{7}$$

Matrizes de covariância para o set $\{y_1, y_2\}$

para $y_0 = A$:

$$M_{CA} = \begin{bmatrix} (0,24 + 0,16 + 0,32) : 3 \\ (0,36 + 0,48 + 0,72) : 3 \end{bmatrix} = \begin{bmatrix} 0,24 \\ 0,52 \end{bmatrix}$$

$$\text{Cov}_A = \begin{bmatrix} \text{var}(y_1) & \text{cov}(y_1, y_2) \\ \text{cov}(y_1, y_2) & \text{var}(y_2) \end{bmatrix} = \begin{bmatrix} 0,0064 & 0,0092 \\ 0,0092 & 0,0336 \end{bmatrix}$$

para $y_0 = B$

$$M_{CB} = \begin{bmatrix} (0,54 + 0,66 + 0,76 + 0,41) : 4 \\ (0,77 + 0,39 + 0,28 + 0,53) : 4 \end{bmatrix} = \begin{bmatrix} 0,5925 \\ 0,3275 \end{bmatrix}$$

$$\text{Cov}_{CB} = \begin{bmatrix} \text{var}(y_1) & \text{cov}(y_1, y_2) \\ \text{cov}(y_1, y_2) & \text{var}(y_2) \end{bmatrix} = \begin{bmatrix} 0,022892 & -0,009758 \\ -0,009758 & 0,037422 \end{bmatrix}$$

Given $\{y_3, y_4\}$ tables:

$$P(y_3=1 \cap y_4=1 | A) = \frac{1}{3}$$

$$P(y_3=0 \cap y_4=0 | A) = 0$$

$$P(y_3=0 \cap y_4=1 | A) = \frac{1}{3}$$

$$P(y_3=1 \cap y_4=0 | A) = \frac{1}{3}$$

$$P(y_3=1 \cap y_4=1 | B) = 0$$

$$P(y_3=0 \cap y_4=0 | B) = \frac{1}{2}$$

$$P(y_3=0 \cap y_4=1 | B) = \frac{1}{4}$$

$$P(y_3=1 \cap y_4=0 | B) = \frac{1}{4}$$

For $\{y_s\}$ tensor:

$$P(y_s = 0 | A) = \frac{1}{3}$$

$$P(y_s = 1 | A) = \frac{1}{3}$$

$$P(y_s = 2 | A) = \frac{1}{3}$$

$$P(y_s = 0 | B) = \frac{1}{4}$$

$$P(y_s = 1 | B) = \frac{1}{2}$$

$$P(y_s = 2 | B) = \frac{1}{4}$$

$$1. b.) \quad y = \arg \max_{c_i} \{ P(c_i | \alpha) \} =$$

$$= \arg \max_{c_i} \{ P(c_i) \times P(\alpha | c_i) \}$$

— para x_8 temos:

$$y = \arg \max_{c_i} \{ P(A) \times P(\alpha_8 | A), P(B) \times P(\alpha_8 | B) \}$$

$$P(\alpha_8 | A) = \text{normalA.PDF}(0,38; 0,52) \times P(y_3=0 \cap y_4=1 | A) \times P(y_5=0 | A) =$$

$$= 0,98470476 \times \frac{1}{3} \times \frac{1}{3} = 0,10947754$$

$$P(\alpha_8 | B) = \text{normalB.PDF}(0,38; 0,52) \times P(y_3=0 \cap y_4=1 | B) \times P(y_5=0 | B) =$$

$$= 1,262359 \times \frac{1}{4} \times \frac{1}{4} = 0,122648$$

$$y = \arg \max_{c_i} \left\{ \frac{3}{7} \times 0,10947754, \frac{4}{7} \times 0,122648 \right\} =$$

$$= \arg \max_{c_i} \{ 0,046890702, 0,070084607 \} = \text{classe B} //$$

— para x_9 temos:

$$y = \arg \max_{c_i} \{ P(A) \times P(\alpha_9 | A), P(B) \times P(\alpha_9 | B) \}$$

$$P(\alpha_9 | A) = \text{normalA.PDF}(0,42; 0,52) \times P(y_3=0 \cap y_4=1 | A) \times P(y_5=1 | A) =$$

$$= 0,4030709859 \times \frac{1}{3} \times \frac{1}{3} = 0,0447857$$

$$P(\alpha_9 | B) = \text{normalB.PDF}(0,42; 0,52) \times P(y_3=0 \cap y_4=1 | B) \times P(y_5=1 | B) =$$

$$= 1,728570587 \times \frac{1}{4} \times \frac{1}{2} = 0,216077323$$

$$y = \arg \max_{c_i} \left\{ \frac{3}{7} \times 0,0447857, \frac{4}{7} \times 0,216077323 \right\} =$$

$$= \arg \max_{c_i} \{ 0,019193871, 0,127469327 \} = \text{classe B} //$$

1.c) For ML, sabemos que $P(A|x) \sim P(x|A)$, assim:

$$P(A|x_8) \approx P(x_8|A) = P(y_1=0,38 \cap y_2=0,52|A) \times P(y_3=0 \cap y_4=1|A) \\ \times P(y_5=0|A) = 0,9847 \times \frac{1}{3} \times \frac{1}{3} = 0,1094$$

$$P(A|x_9) \approx P(x_9|A) = P(y_1=0,42 \cap y_2=0,59|A) \times P(y_3=0 \cap y_4=1|A) \\ \times P(y_5=1|A) = 0,4037 \times \frac{1}{3} \times \frac{1}{3} = 0,04479$$

if $\theta < 0,1717$, we will predict $y_5 = B$ for x_8 and $y_5 = B$ for x_9 , meaning that our accuracy will be 0,5;

if $\theta > 0,4716$, we will predict $y_5 = A$ for x_8 and $y_5 = A$ for x_9 , meaning that our accuracy will also be 0,5;

if θ is between 0,1717 and 0,4716, we will predict $y_5 = A$ for x_8 and $y_5 = B$ for x_9 , which will represent an accuracy of 1.

By analyzing the previously presented information, we conclude that the testing accuracy will be optimized by thresholds between 0,1717 and 0,4716.

2. a)

Dataset after binarization of Y_2

D	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	
X_1	0,24	0	1	1	0	A	Fold 1
X_2	0,76	0	1	0	1	A	
X_3	0,32	1	0	1	2	A	
X_4	0,54	0	0	0	1	B	Fold 2
X_5	0,66	0	0	0	0	B	
X_6	0,76	0	1	0	2	B	
X_7	0,47	1	0	1	1	B	Fold 3
X_8	0,38	1	0	1	0	A	
X_9	0,42	1	0	1	1	B	

2. b.) Hamming distance ($K=3$)

H	x_1	x_2	x_3	x_4	x_5	x_6
x_7	4	4	2	2	3	4
x_8	2	4	1	4	3	5
x_9	4	4	2	2	3	4

For each of the test-observations we'll choose the three ~~closest~~ closest neighbours (since $K=3$)

- $x_7 \rightarrow x_3, x_4, x_5$
- $x_8 \rightarrow x_1, x_3, x_5$
- $x_9 \rightarrow x_3, x_4, x_5$

$$\hat{z}_7 = \frac{\frac{1}{2} \times 0,32 + \frac{1}{2} \times 0,54 + \frac{1}{3} \times 0,66}{\frac{1}{2} + \frac{1}{2} + \frac{1}{3}} = 0,4875$$

$$\hat{z}_8 = \frac{\frac{1}{2} \times 0,24 + 1 \times 0,32 + \frac{1}{3} \times 0,66}{\frac{1}{2} + 1 + \frac{1}{3}} = 0,3600$$

$$\hat{z}_9 = \frac{\frac{1}{2} \times 0,32 + \frac{1}{2} \times 0,54 + \frac{1}{3} \times 0,66}{\frac{1}{2} + \frac{1}{2} + \frac{1}{3}} = 0,4875$$

$$MAE = \frac{\sum_{i=7}^9 |z_i - \hat{z}_i|}{K} = \frac{|0,41 - 0,4875| + |0,38 - 0,36| + |0,42 - 0,4875|}{3} = 0,055 //$$

Homework 2

Group Number: 112

Group Members:

- Duarte Rebelo de São José: ist1103708
- Tiago Ramos da Rocha Neves Caixinha: ist1102437

Imports and data loading

```
from sklearn.model_selection import StratifiedKFold
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from scipy import stats
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
data = arff.loadarff('../column_diagnosis.arff')
df = pd.DataFrame(data[0])
df['class'] = df['class'].str.decode('utf-8')

X = df.drop(columns=['class'])
y = df['class']
```

Problem 1

Generating boxplots

```
kn_classifier = KNeighborsClassifier(n_neighbors=5)
nb_classifier = GaussianNB()

skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)

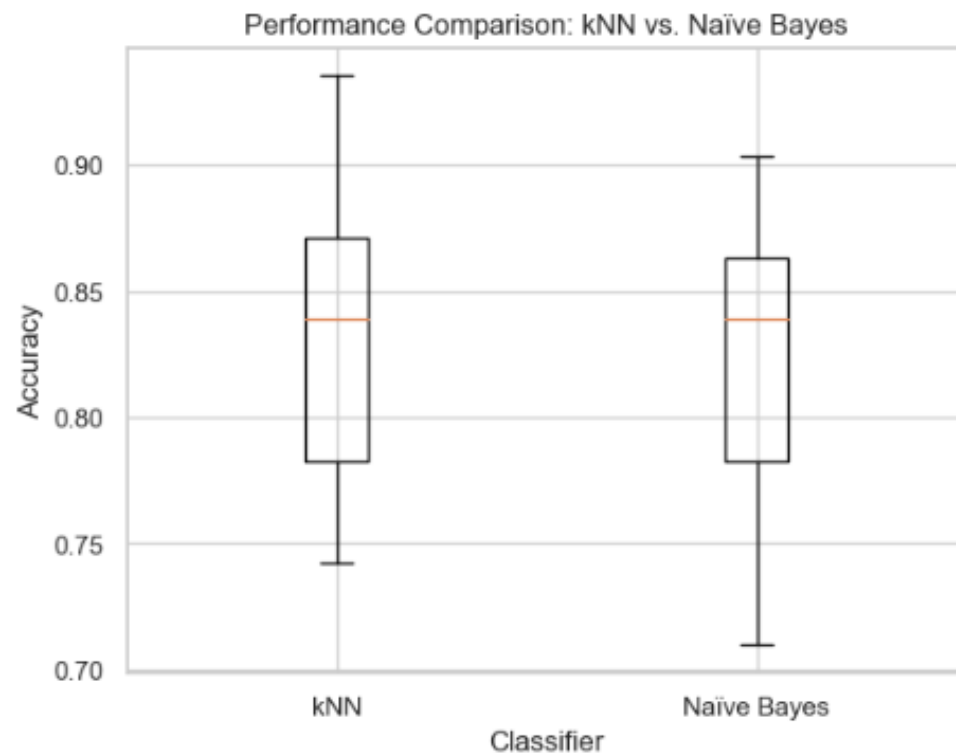
knn_scores = []
nb_scores = []

for train_index, test_index in skf.split(X, y):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    kn_classifier.fit(X_train, y_train)
    kn_score = kn_classifier.score(X_test, y_test)
    knn_scores.append(knn_score)

    nb_classifier.fit(X_train, y_train)
    nb_score = nb_classifier.score(X_test, y_test)
    nb_scores.append(nb_score)

plt.boxplot([knn_scores, nb_scores], labels=["kNN", "Naïve Bayes"])
plt.ylabel("Accuracy")
plt.xlabel("Classifier")
plt.title("Performance Comparison: kNN vs. Naïve Bayes")
plt.show()
```



Testing the given hypothesis

```
t_stat, p_value = stats.ttest_rel(knn_scores, nb_scores)

alpha = 0.05

if p_value < alpha:
    print("The given hypothesis should be rejected.")
else:
    print("The given hypothesis should be accepted.")
```

The given hypothesis should be accepted.

Problem 2

Training the decision tree with various depth limits

```
unique_classes = np.unique(y)

knn_1 = KNeighborsClassifier(n_neighbors=1)
knn_5 = KNeighborsClassifier(n_neighbors=5)

num_classes = len(unique_classes)
cumulative_cm_1 = np.zeros((num_classes, num_classes))
cumulative_cm_5 = np.zeros((num_classes, num_classes))

skf = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)

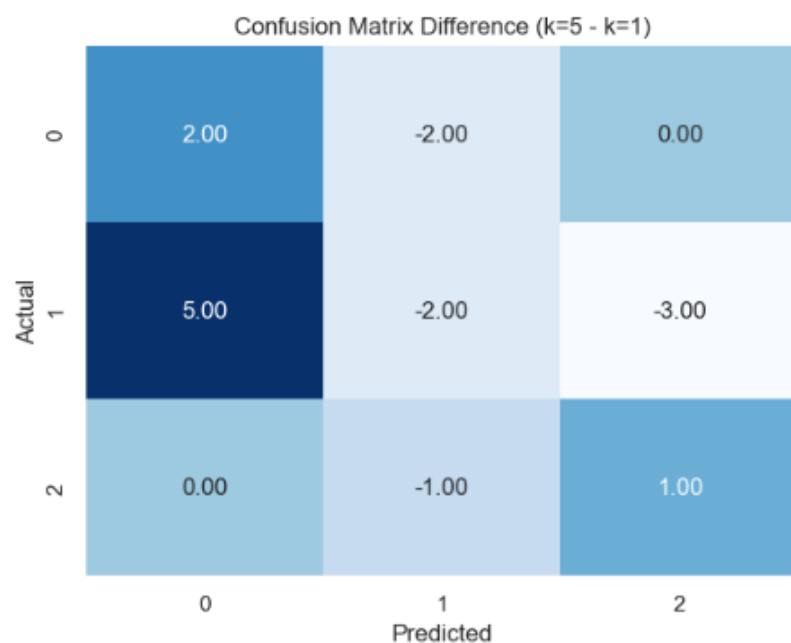
for train_index, test_index in skf.split(X, y):
    X_train, X_test = X.iloc[train_index], X.iloc[test_index]
    y_train, y_test = y.iloc[train_index], y.iloc[test_index]

    knn_1.fit(X_train, y_train)
    y_pred_1 = knn_1.predict(X_test)
    cm_1 = confusion_matrix(y_test, y_pred_1, labels=unique_classes)
    cumulative_cm_1 += cm_1

    knn_5.fit(X_train, y_train)
    y_pred_5 = knn_5.predict(X_test)
    cm_5 = confusion_matrix(y_test, y_pred_5, labels=unique_classes)
    cumulative_cm_5 += cm_5

cm_diff = cumulative_cm_5 - cumulative_cm_1

sns.heatmap(cm_diff_float, annot=True, fmt='.2f', cmap='Blues', cbar=False)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix Difference (k=5 - k=1)")
plt.show()
```



Comment

The heatmap shows the differences in confusion matrices between kNN with k=5 and k=1.

Problem 3

Identifying three possible difficulties of Naïve Bayes

Naïve Bayes is a simple and effective classifier, but it may face difficulties in certain scenarios:

1. **Assumption of Independence:** Naïve Bayes assumes that features are conditionally independent given the class label. In real-world datasets, this assumption may not hold true, leading to suboptimal performance when features are correlated.
2. **Continuous Features:** Naïve Bayes with Gaussian assumption assumes that continuous features follow a Gaussian distribution. If the data does not meet this assumption, the model may not capture the underlying distribution accurately.
3. **Data Imbalance:** Naïve Bayes can be sensitive to class imbalances. If one class has significantly more instances than the others, the model may be biased towards the majority class and perform poorly on minority classes.